Triangulation of an elevation surface structured by a sparse 3D grid

Van Sinh NGUYEN
International University of HCMC
School of Computer Science and Engineering
Quarter 6, Linh Trung, Thu Duc, HCM, Vietnam
Tel: 84.8.37.24.42.70 - ext 32.25
Email: nvsinh@hcmiu.edu.vn

Alexandra BAC
Aix-Marseille University
CNRS, Laboratory LSIS
UMR 7296, Marseille, France
Tel: 33.4.91.82.85.32
Email: alexandra.bac@univ-amu.fr

Marc DANIEL
Aix-Marseille University
CNRS, Laboratory LSIS
UMR 7296, Marseille, France
Tel: 33.4.91.82.85.25
Email: marc.daniel@univ-amu.fr

***Abstract*—This paper proposes a method to triangulate an elevation surface structured in a sparse 3D grid by using a fast search algorithm based on the 2D Delaunay triangulation. After projecting the 3D point clouds onto a natural 2D grid in the x, y plane, we triangulate the surface (actually, we compute a Delaunay triangulation of the 2D point cloud taking advantage of its regular structure). The main novelty of our approach is that the neighboring points of an edge e_i are searched in a rectangle supported by the edge e_i under consideration. The obtained results show that our method could process very fast, the initial shape of the surface is well preserved, and the topology of the output triangular mesh approximates the input surface of 3D point clouds.**

Keywords: Surface reconstruction, 3D point cloud, 2D Delaunay triangle, voxel traversal search.

I. Introduction

Surface triangulation of a 3D object is a fast and efficient way to reconstruct such an approximating surface in the 3D space. Therefore, meshing a 3D point cloud has been studied widely in the field of geometric modeling [18]. The goal is to reconstruct a surface of triangular mesh from a given 3D point cloud, such that the triangular mesh must approximate the original shape of the input surface.

Various methods are available for generating a triangular mesh from a 2D or 3D point cloud [4], [9], they can be applied on both open and close surfaces. The computing time of algorithms and the quality of generated triangular meshes are two essential objectives that the researchers are facing.

Our purpose is to build an optimal geological triangulated surface in order to get the best simulation of the oil reservoir [1]. The problem in our study comes from the seismic data, with a very large number of 3D points (that can reach several millions of points). Therefore, the necessary time to triangulate these data points with "classical" methods can be very high (and hence unacceptable in terms of time and memory). For this reason, the input data are first structured in a sparse 3D volume as defined by Philippe Verney [3]. In order to preserve the shape of the surface, the first step is extracting and simplifying the boundary of the surface [5]. The proposed method for simplifying the surface inside its boundary [6] is an important step for the further meshing step; indeed, simplification produces smaller data sets whose density varies regularly over the surface. The work in this paper is the next step in which we propose a method to triangulate this surface by using a fast search algorithm [11] based on the 2D Delaunay triangulation.

The remainder of this paper is organized as follows: we first present the existing methods for triangulating a surface of 2D and 3D point clouds in section II. Our method is presented in detail in section III. Section IV is the implementation. The results, discussion and evaluation are presented in section V and VI. The last section (VII) is our conclusion.

II. Related works

The methods for triangulating a surface of point clouds have been studied on both 2D and 3D. Some existing methods for meshing a triangular surface of a 3D point set have been proposed in [2], [7], [8], [10]. Other existing approaches for building a triangular mesh of 2D points have been suggested in [9], [14]–[16], [19]. Most of these methods are classified into three following categories: *Contour tracing* approach (or called implicit surface approach) is a method which produces approximating rather than interpolating surface [7]. *Region growing* approach always starts from a seed triangle in a local region of a point set; then considers a new point in the existing region boundary to connect and create a new triangle. The process will stop until all points have been touched [8]. *Sculpting-based* approach is also called Delaunay-based approach. Fang et. al. [15] proposed a method for triangulating the 2D data points based on a uniform grid structure. This method can process very fast, but triangulates the convex hull of the surface. Domiter et. al. [16] introduced a method for 2D constrained Delaunay triangulation built upon sets of points and constraining edges. The method uses a sweep-line paradigm combined with a Lawson's legalization. However, the computing time is expensive for checking the Delaunay criterion of edge flipping. In 3D, many methods are based on the computation of an approximate Delaunay mesh. Kazhdan et. al. [12] suggested a method for surface reconstruction from the oriented points of 3D point clouds based on the Poisson formulation. The method falls into the category of contour tracing approach using an implicit function. The proposed method of Bernardini et. al. [8] (called Ball Pivoting

Algorithm: BPA) is considered as an example of the region growing technique. One of the disadvantages is that the ball radius ρ does not adapt to local point density; many holes may be generated; and the quality of the mesh depends on the radius ρ defined by user. The method proposed by Hong-Wei et. al. [10] overcomes the user's specified ball-radius in the BPA method [8], but the computation is complex. The above methods can be applied to triangulate a 2D or 3D point cloud, with or without organization. They generate a triangular mesh approximating the original model of the input data. In our work, we take the advantage of data structure in the 3D grid for building a method to obtain a faster computation. The next section will detail our method.

III. Methods for triangulating the surface

A. Overview

Our method is based on the 2D Delaunay triangulation of the projected point cloud. After projecting the 3D point cloud onto a 2D grid, we compute a Delaunay triangulation of the 2D point cloud. Starting from the first Delaunay triangle T, the next adjacent triangle T' is then created between an edge e_i of T and a neighboring point based on the voxel traversal search [11] within a rectangle, on one side of e_i. At each step, the new edges of the created triangle T' are inserted in a pool and the process is iterated starting from T' until the pool is empty.

B. Concepts, notation and definition

In order to clarify the context, let us introduce some concepts and notations used in the sequel. The boundary of the surface has been determined in [5]; it is a polyline connecting the boundary edges $e(v_i, v_{i+1})$ with the numbering of the polyline (each point belongs to exactly one pixel). Therefore, we will freely use either the continuous or discrete notations:

- Given an edge $e = (p_1, p_2)$, there exist exactly two pixels v_1, v_2 such that $p_i \in v_i$ $(i = 1, 2)$.
- p_b: a boundary point;
- e_b: a boundary edge, $e_b = (p_{b1}, p_{b2})$.
- The continuous size of e is, as usual given by $\|p_2 - p_1\|$, we will denote it by $\|e\|$.
- The discrete size of e (denoted by $|e|$) as the discrete distance between v_1 and v_2 ($\|v_2 - v_1\|_\infty$ in [5]).
- Given a triangle (p_2^i, p_1^i, p_3^i) and the edge $e_i = (p_1^i, p_2^i)$, a point p_4^i is called opposite to p_3^i with respect to e_i if it belong to a vertex of a triangle $\triangle(p_1^i, p_2^i, p_4^i)$ and opposite to a neighboring point p_3^i by edge e_i.
- $k = |e_i|$: the discrete size of e_i.
- $Re(e_i)$: a rectangle, supported by e_i.

C. Building a seed triangle

The existing methods for creating a first (seed) triangle are introduced in [8], [10], [15], [19]. Normally, most of these methods always start from the first point p_i (it can be a random choice, a middle point or a point with a special value); then a point p_j close to p_i is chosen in order to create the first edge e_{ij}. The next step is finding a neighboring point p_k such that p_k satisfies a given condition, e.g. p_i, p_j, p_k are formed a Delaunay triangle. This triangle is then considered as a seed triangle. In our case, we start from the left-most boundary point p_{b1} to find a closest neighboring boundary point p_{b2} on the boundary. Then, we find a neighboring point p_3^i of e_b such that p_3^i must satisfy the Delaunay criterion. Therefore, the triangle $\triangle(p_{b1}, p_{b2}, p_3^i)$ is our seed triangle T_{first}. Let us give a proof that a Delaunay edge has always been existed in our data:

Property: Let p_1 be a point belonging to a set of vertices V ($p_1 \in V$). Let p_2 be the closest neighbor of p_1. The edge p_1p_2 is a Delaunay edge.

Proof: We orient edge p_1p_2 from p_1 to p_2 (see figure 1a); H is a half space on the right side of p_1p_2. The vertices in H are sorted by an increasing order of angles ($v_i \rightarrow \alpha_i$). We choose v_i with the largest angle $\widehat{p_1\alpha_i p_2}$ at p_3 such that the intersection between circum($p_1p_2p_3$) and H is empty. As p_2 is closest from p_1, $\|p_1p_3\| \geq \|p_1p_2\|$; and the arc($p_1, p_2, p_3$) is larger than the half circle $\Leftrightarrow \widehat{p_1p_2p_3} < 90^o$. If there is a point p_4 which belongs to circum($p_1p_2p_3$), on the left of p_1p_2 (see figure 1b); then, $\|p_1p_4\| < \|p_1p_2\|$. This is impossible because p_2 is the closest point to p_1. Hence, there is no such point p_4; and p_1p_2 is a Delaunay edge. ■

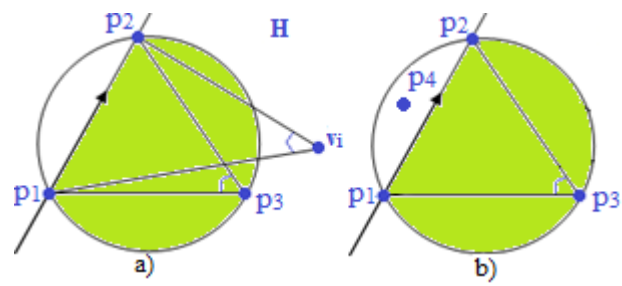

Fig. 1: Determination of the first Delaunay edge.

After building the seed triangle T_{first}, we insert its three edges into the EdgePool for the next iteration step. In the next section, we describe the condition of non-intersection for generated triangles in the triangular mesh.

D. Searching conditions in one side of an edge

For each edge e_i of the previous triangle (e.g. $\triangle(p_1^i, p_2^i, p_4^i)$), we need to find a neighboring point p_3^i to form a new adjacent triangle $\triangle(p_2^i, p_1^i, p_3^i)$. In order to avoid creating intersecting triangles, we require the following conditions for p_3^i. The first condition is that the three points (p_1^i, p_2^i, p_3^i) are not aligned. The second condition is that p_3^i and p_4^i must be on either side of e_i.

E. Delaunay Criterion

In this section, we describe a fast way to compute the Delaunay criterion (testing the emptiness of a circumcircle). For each e_i, we find an optimal neighboring point p_3^i in one side of $e_i = (p_1^i, p_2^i)$ to form a triangle $T(p_1^i, p_2^i, p_3^i)$. Then, we check the emptiness of the circumcircle C going through p_1^i, p_2^i, p_3^i. If C is empty, then T satisfies the criterion of the Delaunay triangle.

F. Neighboring points search based on the voxel tracing

Our goal is to triangulate a surface such that all its triangles are as equilateral as possible. Therefore, the neighboring point p_3^i will be searched as close as possible to the position of the third vertex of the equilateral triangle lying on e_i. We apply the voxel traversal algorithm of Amanatides and Woo (AW) introduced in [11] to search a neighboring point p_3^i of e_i within a rectangle $Re(e_i)$, size k (we denote by k the discrete size of e_i (that is, as explained previously, the discrete distance between the pixels containing the extremities of e_i: see Fig.2).

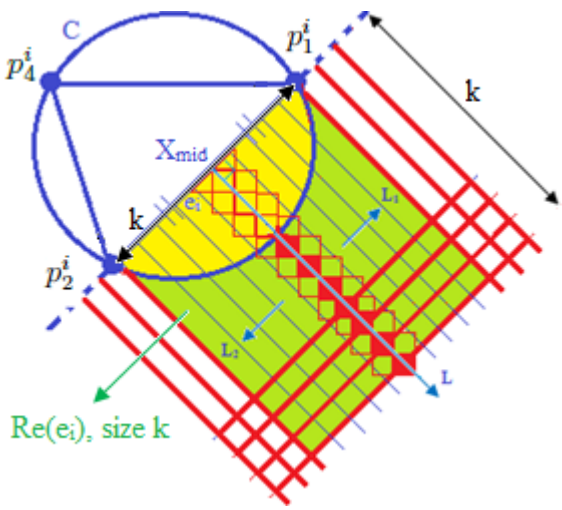

Fig. 2: The voxels traversal search

We use the following notations (illustrated on figure 2) to denote and compute in our algorithms:

- AW algorithm will be described in a very generic way (with indices i ranging in $\{1..n\}$ n being the dimension). In our case, $n = 2$.
- $X[i]$ with $i \in \{1..n\}$: coordinates of the current voxel.
- L: the discrete mediatrix (the equidistant continuous voxel set) of e_i up to a distance k of X_{mid} (the voxel in the middle position of edge e_i).
- L': the successive dilations of L.
- $Step[i]$ with $i \in \{1..n\}$: for each coordinate, direction of L (+1 if the positive direction along the ith axis, -1 otherwise) will indicate the coordinates of next point along each axis.
- $tMax[i]$ with $i \in \{1..n\}$: time at which the ray crosses the output plane along the ith axis.
- $tDelta[i]$ with $i \in \{1..n\}$: time to cross a voxel along the ith axis.

For each edge e_i, we use the AW algorithm [11] to determine L under the form:

$$U + t.V \quad (t \in \mathbb{R}^+) \tag{1}$$

where, U: the initial point of L; in our case, $X_{mid}(x_{mid}, y_{mid})$; V: the direction of L (orthogonal to e_i and oriented opposite to (p_2^i, p_4^i); and t: the time. In the next section, we will compute the discrete mediatrix L and the successive dilations L' in order to find an optimal neighboring point p_3^i of e_i.

1) Computing the discrete mediatrix L: We execute AW algorithm to compute the discrete mediatrix up to a distance k of X_{mid} (and starting from this point). At each step, given the current voxel $X(x, y)$, we decide whether to increase the coordinates of x or y or both x, y to find the next voxel. This set of voxels is stored in a list.

Initialization. We compute:

- X_{mid}: the initial voxel of L.
- V: we have $e_i = p_1^i - p_2^i = (x, y)$, let us set $v = (-y, x)$ the orthogonal direction and $v' = p_4^i - p_2^i$ and edge of the previous triangle. If $v \times v' > 0$, then we set $V = -v$ otherwise, $V = v$ (that is V is the direction of the half-line orthogonal to e_i and opposite with respect to the previous triangle (p_1^i, p_2^i, p_4^i)).
- For this voxel, we compute $tMax[i]$; $tDelta[i]$ and $Step[i]$ (which are static and will be constant along the algorithm).

Iterations. At each step:

- Let i be the axis such that tMax[i] is minimal
- Update X (by adding $Step[i]$ to its ith coordinate)
- Update $tMax$

We use the algorithm (Algorithm 1) to compute the discrete mediatrix L as follow:

Algorithm 1 AWLine(e_i, k, rl)

```
//pointL: the endvoxel of L (size k from X_mid)
compute: X_mid, V; compute: pointL;
compute: tMax[i], tDelta[i], Step[i];
repeat //start at X = X_mid
    rl.insert(X);   //save current voxel
    for each dimension i do
        tMax[i]+ = tDelta[i];
        X[i]+ = Step[i];
    end for //X is next crossed voxel (our new voxel)
until X = pointL
rl.insert(pointL);
```

2) Computing the successive dilations L′: All points lying on the discrete mediatrix of e_i are tested with the way determined in section E in order to determine if they satisfy the Delaunay criterion. If none of them coincides, we compute the successive dilations L' of the discrete line L and test the vertices they may contain. This dilation is constituted of two discrete lines (called L_1 and L_2: see figure 2). The vertices on L_1 and L_2 are determined based on the neighboring vertices of each current vertex with its x, y coordinates $X(x, y)$ on L. The successive dilations $(L_1 \cup L_2)$ are explored until $Re(e_i)$ has been fully explored. In this case, if there is no point in $Re(e_i)$, we will explore the successive neighborhoods of $Re(e_i)$ (in one side of e_i, start from voxel $pointL + 1$) until finding an appropriate point. In the next section, we will present our algorithm for searching a point based AW algorithm.

3) Algorithm based on the voxel traversal search: In this section, we describe our algorithm based on the voxel traversal search. We note l, the list containing voxels crossed with the ray line L; similarly, l_1, l_2 for L_1 and L_2 respectively. The algorithm includes the following two steps:

- First step: we compute the discrete mediatrix of e_i by calling the function $AWLine(e_i, k, rl)$ which return the list of voxels stored in rl; then, the list rl is assigned to l. For each non-empty voxel $v \in l$ (i.e. a neighboring point: p_3^i), if it satisfies the Delaunay criterion ($Circumcircle(p_2^i, p_1^i, p_3^i)$), we stop searching. Otherwise, we go to step 2.
- Second step: we repeat the process on the neighboring voxels of l on l_1 and l_2 successively. If there is no point p_3^i on the rectangle of edge e_i, we enlarge the searching by iterative dilations of $Re(e_i)$, and stop when a neighboring point p_3^i satisfying the Delaunay criterion is found.

The algorithm below (Algorithm 2) is used to find this neighboring point p_3^i. In this algorithm, each non-empty voxel $v \in Re(e_i)$ is considered as a neighboring point p_3^i.

Algorithm 2 SearchNeighbors(e_i, p_3^i)

```
find ← false, k=1;
while !find do
    AWLine(e_i, k, rl); //Step1: compute L.
    l_1 = l_2 = l ← rl;
    for each voxel v ∈ l do
        if v != empty and Cir(e_i, v) == true then
            find ← true;
        else
            mark_v = true;
        end if
    end for
    while inside the Re(e_i) do  //Step2: compute L'
        compute l'_1, the neighbor voxels of l_1
        for each voxel v ∈ l'_1 do
            if v != empty and Cir(e_i, v) == true then
                find ← true;
            else
                mark_v = true;
            end if
        end for
        compute l'_2, the neighbor voxels of l_2
        for each voxel v ∈ l'_2 do
            if v != empty and Cir(e_i, v) == true then
                find ← true;
            else
                mark_v = true;
            end if
        end for
        l_1 ← l'_1; l_2 ← l'_2;
    end while
    dilation from Re(e_i);
end while
```

G. Triangulating a surface

Following the previous analysis, we present in this section our algorithm for constructing a triangular mesh of an elevation surface defined by a sparse 3D grid. The corresponding algorithm (Algorithm 3) is used as follow:

Algorithm 3 MeshGenerating(S)

```
start from the first Delaunay boundary edge;
create the first Delaunay triangle T_first;
insert all edges of T_first into a list EdgePool;
while EdgePool != empty do
    for each edge e_i(endpoints: p_1^i, p_2^i) do
        if e_i != visited then
            SearchNeighbors(e_i, p_3^i);
            T_next = △(p_1^i, p_2^i, p_3^i);
            update new edges of T_next into the EdgePool;
            e_i = visited;
            T_first ← T_next;
        end if
    end for
end while
```

IV. Implementation

In this section, we implement our proposed method for triangulating an elevation surface of 3D point clouds based on the voxel traversal search. Our algorithms are programmed and integrated in Meshlab [20] as a plug-in. Therefore, we can test the running time between our method and the existing method in the Meshlab (ball pivoting). However, it is difficult to evaluate exactly the comparison of the time between the methods (that depends on the similarity of algorithms and data structure between these methods). For this reason, we also introduce and execute our adding method for finding a neighboring point p_3^i of e_i which can be applied in our context to compare exactly the processing time with the voxel traversal search. This method is based on computing the compactness of a triangle introduced in [2], [13]. As presented in section III, instead of using the voxel traversal search, we now used the formula of Guéziec [13]. We compute the compactness of a triangle among triangles $\triangle(p_1^i, p_2^i, p_l)$ where ($l = 1..v_i$) with v_i is the number of neighboring points in one side of $e_i(p_1^i, p_2^i)$. The formula is as follows:

$$C = \frac{4\sqrt{3}A}{l_0^2 + l_1^2 + l_2^2} \tag{2}$$

Where C denotes the compactness of triangle T; A is area of T; l_0, l_1 and l_2 are respectively the three lengths of each edge of T. We compute all triangles between $e_i(p_2^i, p_1^i)$ and its neighboring points based on equation (2) to find a point p_l of a compactness triangle within $Re(e_i)$. If this point is found, we then check the Delaunay criterion and form the triangle $\triangle(p_2^i, p_1^i, p_l)$. We implement both methods (voxel traversal search: VTS and computing the compactness: CTC) to test and compare the processing time between them and the existing method in Meshlab [20], Ball pivoting: BP.

V. Results

In this section, we also show the initial shape of the surface is well preserved. The processing times are presented in table 1. In this table, we use the same input surfaces to test with the three methods. Both VTS and CTC methods are integrated and tested (one by one) on the same computer. They are only different from their ways to find a neighboring point. The processing time is a bit different between them (we will explain the reason why in the section VI). The BP method generates a very good mesh, but the processing time is higher than our methods. If the number of input points increases, the running time is far from our method.

Input points	CTC Output faces (Time: ms)	VTS Output faces (Time: ms)	BP Output faces (Time: ms)
15626	29871/14631	29871/13360	29990/18984
32402	61073/30911	61073/28984	58030/67871
60511	115828/68797	115828/63223	110982/225037
68956	125463/112117	125463/93246	118273/268276
98231	181851/194682	181851/162451	174582/859341
148317	266187/246435	266187/218713	267842/2017632
886639	1618624/1473188	1618624/1307466	1601158/11951403

TABLE I: Comparison of the processing times between the methods. We use the same input data points and run on the same computer (Intel 2CoreDue, 2GB of Ram).

The exterior boundary of S and surface S has been processed in the previous works [5], [6]. Therefore, the number of boundary points and the density of them depend on the previous processing. For these reasons, the number of triangular faces is a bit different from the classical triangulation algorithms. In order to evaluate the quality of our triangulated surface with respect to its initial shape, we use the Metro tool [17]. After triangulating the surfaces, the initial shape of these surfaces is well preserved (see Fig.3 and 4). In conclusion, the initial shape of the BP method is well preserved after triangulating while consuming the running time for the surface triangulation. Our method VTS obtains the good results on both processing time and approximation errors.

VI. Discussion and evaluation

In this section, let us give some discussions and evaluations about the processing time of our method. We triangulate the surface based on the 2D Delaunay triangulation following the category of region-growing. Our method has two advantages compared to existing methods. The first and most important point is that we design the rectangle and our particular search in order to speed up the Delaunay triangulation. The neighboring point p_3^i is always searched in one side of an edge e_i with two endpoints (p_1^i, p_2^i) within a rectangle $Re(e_i)$. In the CTC method, for each edge e_i, we have to compute all neighboring points q_i (with i, the number of neighboring points of e_i) to find the best one for creating the compactness triangle. Therefore, the complexity of this computation is always at most $N \times i$ (where N is the number of points of the surface) for iteration. In the VTS method, we base on the AW algorithm [11] to save the searched time for finding p_3^i. We aim to find the point that is close to the position of the third point of an equilateral triangle created by e_i (this point is always located on the non-empty voxel crossed on the ray L, if it exists).If there is no point on L, we dilate from L until finding an appropriate point. For this reason, we do not need to check all neighboring points as computed in the CTC method. Comparing to the CTC method, the processing time of our method is faster, but no guarantee exists for the worst case if an important dilation is required. The BP method computes a triangle mesh interpolating a given point cloud entirely in 3D. In order to find a neighboring point, it has to test all points by pivoting around an edge until finding the best point; therefore, the processing time is higher. While the method of Fang et. al. [15] triangulates the convex hull of the surface. The second advantage is that our surface is triangulated after a simplification step [6]. In fact, for each edge e_i, there is only a few neighboring points p_3^i in one side of e_i within $Re(e_i)$. Therefore, the processing time is very fast, as well as the initial shape of the surfaces is well preserved.

VII. Conclusion

In this paper, we present our method for triangulating the surface of 3D point clouds defined in the 3D grid. We applied the ideas of the methods introduced in [11], [15] to build our method. The main idea of our method based on the 2D Delaunay triangulation is to use the voxel traversal search. The advantages of our method are presented in the evaluation and discussion. The obtained results have shown that the processing times are very fast. After simplifying and triangulating a surface, the triangular faces of this surface have varied automatically from the boundary to the inside of the surface. Therefore, the obtained results are adapted to our goal. However, the fault on the surface is not handled because if two points exist with the same coordinates x, y but different z coordinates, this has been discarded before our work. In the future, we continue to study the methods for refining the meshes in order to obtain an optimal triangular surface.

Acknowledgment

This research work is funded by the project "Constructing the model of triangular grid surface from 3D point clouds bases on a fast search algorithm" at HCMIU 2014-2015. We would like to thank HCMIU for the fund and the reviewers for their valuable comments.

References

[1] NGUYEN Van Sinh, "3D Modeling of elevation surfaces from voxel structured point clouds extracted from seismic cubes", *PhD Thesis*, Aix-Marseille University, 2013.

[2] N.V.TRAN, "Traitement de surfaces triangulées pour la construction des modèles geologique structuraux", *PhD Thesis*, Université de la Méditerranée, 2008.

[3] Philippe Verney, "Interprétation géologique de données sismiques par une méthode supervisée basée sur la vision cognitive", *PhD Thesis*, École Nationale Supérieure des Mines de Paris, 2009.

[4] Ji Ma, "Surface reconstruction from unorganized point cloud data via progressive local mesh matching", *PhD Thesis*, School of Graduate and Postdoctoral Studies, The Univerity of Western Ontario, 2011.

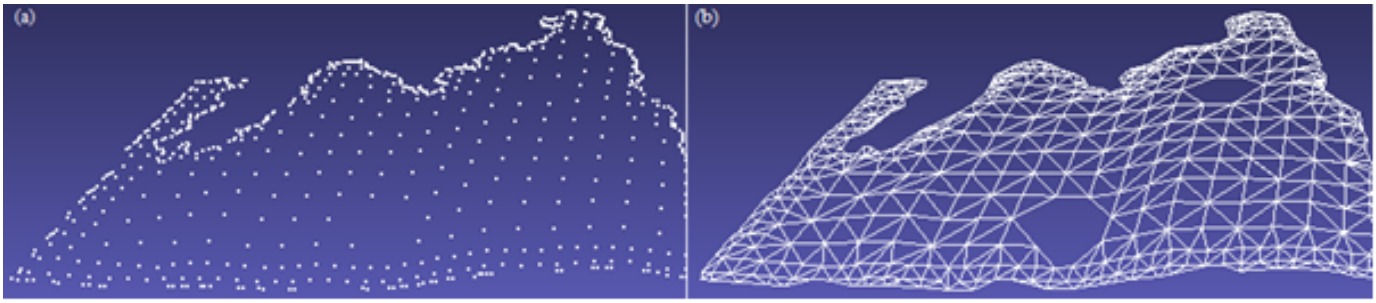

Fig. 3: a) A geological surface of 3D point clouds (232 kb). b) After simplifying by using the elaborate method [6] (cell size = 8, $\partial \leq 0.12$) and triangulating, the size of surface: 18 kb; the approximation error between (a) and (b) is Δ_{max}: 0.020; Δ_{avg}: 0.0006; the triangular faces vary in density from the boundary to the inside of the surface.

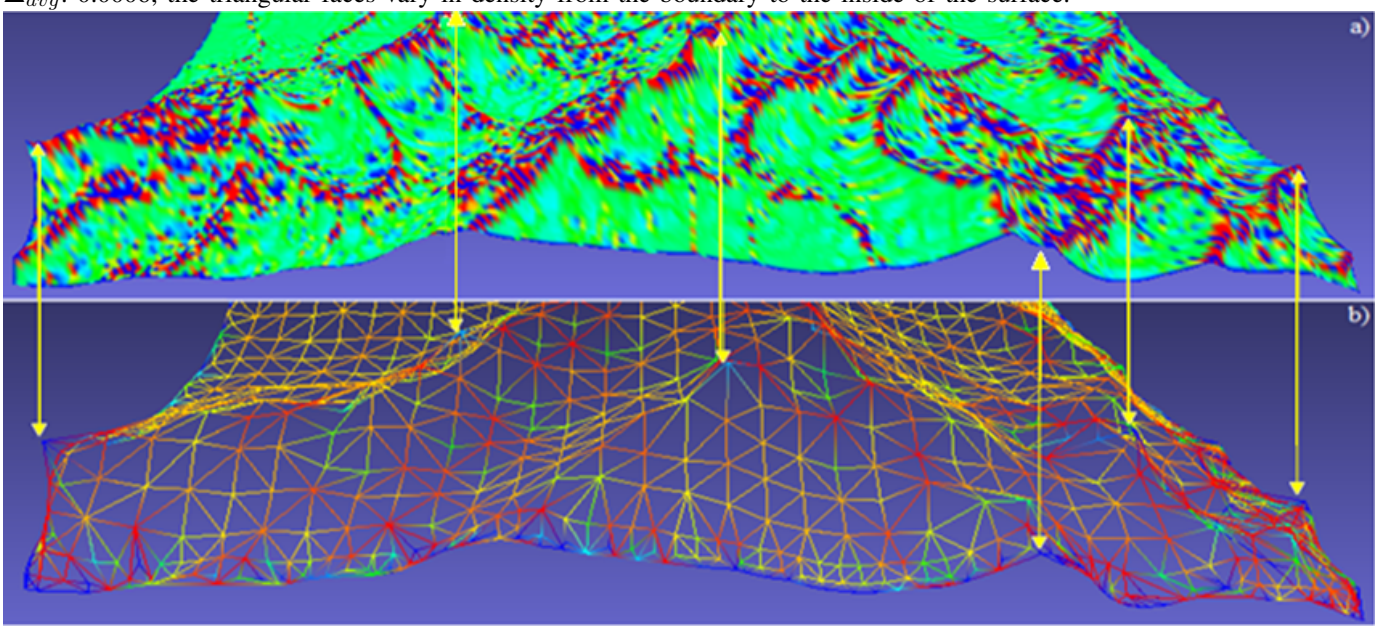

Fig. 4: a) The input surface of 3D point clouds with 2629 kb. b) After simplifying by using the elaborate method [6] (cell size = 8, $\partial \leq 0.09$) and triangulating, the size of surface: 68 kb; the approximation error between (a) and (b) is Δ_{max}: 0.018; Δ_{avg}: 0.002; There are some color points which have shown that the characteristics of the surface are well preserved.

[5] V.S.NGUYEN, A.BAC, M.DANIEL, "Boundary Extraction and Simplification of a Surface Defined by a Sparse 3D Volume", *Proceeding of the third international symposium on information and communication technology SoICT 2012, Pages. 115-124, ACM-ISBN: 978-1-4503-1232-5*, 2012.

[6] Nguyen.V.S, Bac.A, Daniel.M, "Simplification of 3D Point Clouds sampled from Elevation Surfaces", *Simplification of 3D Point Clouds sampled from Elevation Surfaces. 21st International Conference on Computer Graphics, Visualization and Computer Vision WSCG 2013, Pages. 60-69, ISBN: 978-80-86943-75-6*, 2013.

[7] Hugues Hoppe, "Surface Reconstruction from Unorganized Points", *PhD Thesis*, University of Washington, 1994.

[8] F.Bernardini, J.Mittleman, H.Rushmeier, C.Silva, G.Taubin, "The Ball-Pivoting Algorithm for Surface Reconstruction", *IEEE Transactions on Visualization and Computer Graphics, 5(4), Pages. 349 - 359*, 1999.

[9] J.R. Shewchuk, "Triangle: Engineering a 2D quality mesh generator and Delaunay triangulator", *In Applied computational geometry towards geometric engineering, Volume. 1148 of Lecture Notes in Computer Science, Pages. 203-222*, Springer-Verlag, Berlin May, 1996.

[10] H.W.Lin, C.L.Tai, G.J.Wang, "A mesh reconstruction algorithm driven by an intrinsic property of a point cloud", *Computer-Aided Design, 36(1), Pages. 1 - 9*, 2004.

[11] John Amanatides, Andrew Woo, "A Fast Voxel Traversal Algorithm for Ray Tracing", *In Eurographics 87, Pages. 3-10*, 1987.

[12] M.Kazhdan, M.Bolitho, H.Hoppe, "Poisson Surface Reconstruction", *SGP '06 Proceedings of the fourth Eurographics symposium on Geometry processing, ISBN:3-905673-36-3*, 2006.

[13] Andre Guziec, "Surface Simplification Inside a Tolerance Volume", *IBM Research Report RC 20440, NY 10598*, May 1997.

[14] Y.J.Yang, H.Zhang, J.H.Yong, W.Zeng, J.C.Paul, J.Sun, "Contrained Delaunay Triangulation Using Delaunay Visibility", *ISVC 2006, LNCS 4291, Pages. 682-691*, Springer-Verlag Berlin Heidelberg 2006

[15] T.P.Fang, L.A.Piegl, "Delaunay triangulation using a uniform grid", *IEEE Computer Graphics and Applications, Pages. 36-47, 13(3)*, 1993.

[16] V.Domiter, B.Zalik, "Sweep-line algorithm for constrained Delaunay triangulation", *International journal of Geographical Information Science, 22(4), Pages. 449-462*, 2008.

[17] P.Cignoni, C.Rocchini, R.Scopigno, "Metro: Measuring error on simplified surfaces", *The Eurographics Association 1998, 17(2)*, 1998.

[18] M. Botsch, M. Pauly, L. Kobbelt, P. Alliez, B. Lévy, S. Bischoff, C. Rossl, "Geometric Modeling Based on Polygonal Meshes", *ACM SIGGRAPH Course Notes, Pages. 11-159*, 2007.

[19] R.Klein, "Construction of The Constrained Delaunay Triangulation of A Polygonal Domain", *In CAD Systems Development. Springer Berlin Heidelberg, ISBN: 978-3-642-60718-9, Pages. 313-326*, 1997.

[20] ISTI, "Meshlab", *http://meshlab.sourceforge.net/*, 2013.